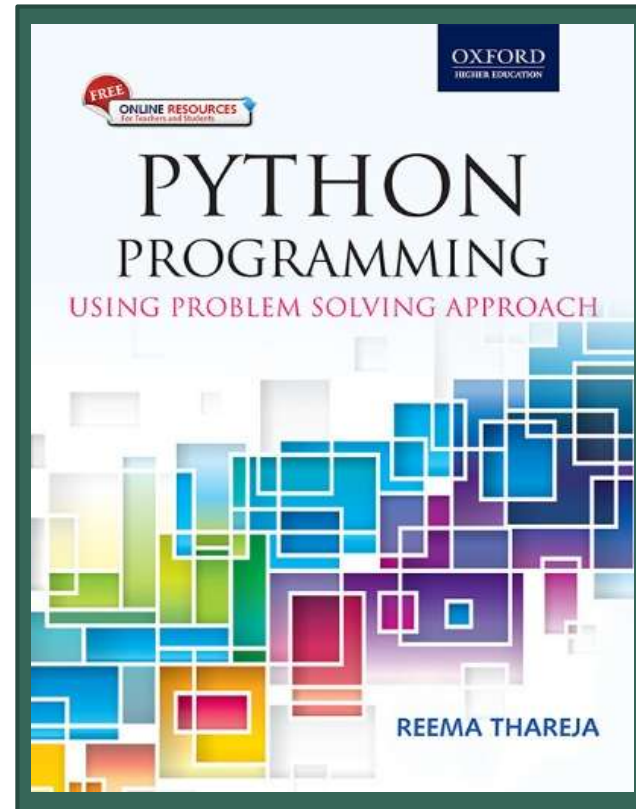


Python Programming

Using Problem Solving Approach

**Reema
Thareja**



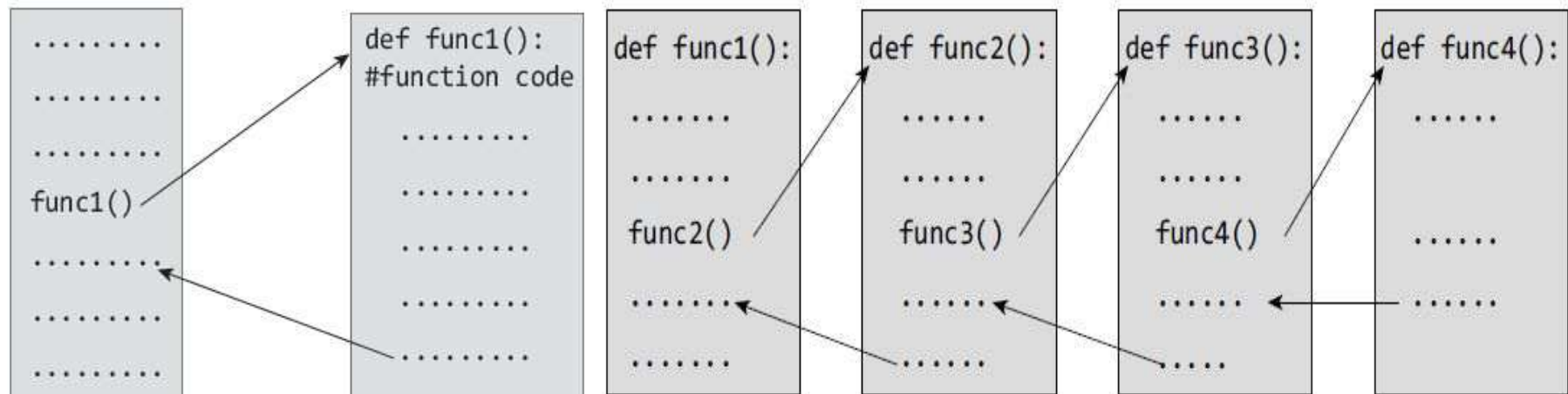
1



Functions and Modules

Functions

Python enables its programmers to break up a program into segments commonly known as functions, each of which can be written more or less independently of the others. Every function in the program is supposed to perform a well-defined task.



Need for Functions

Each function to be written and tested separately.

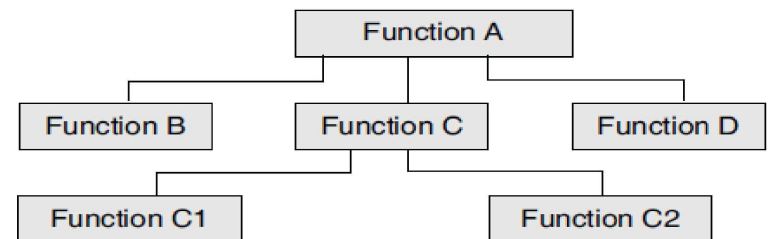
- Understanding, coding and testing multiple separate functions is far easier.

Without the use of any function, then there will be countless lines in the code and maintaining it will be a big mess.

- Programmers use functions without worrying about their code details. This speeds up program development, by allowing the programmer to concentrate only on the code that he has to write.

Different programmers working on that project can divide the workload by writing different functions.

- Like Python libraries, programmers can also make their functions and use them from different point in the main program or any other program that needs its functionalities.



Function Declaration and Definition

- A function, f that uses another function g , is known as the **calling function** and g is known as the **called function** .
- The inputs that the function takes are known as **arguments/parameters** .
- When a called function returns some result back to the calling function, it is said to return that result.
- The calling function may or may not pass parameters to the called function. If the called function accepts arguments, the calling function will pass parameters, else not.
- **Function declaration** is a declaration statement that identifies a function with its name, a list of arguments that it accepts and the type of data it returns.
- **Function definition** consists of a function header that identifies the function, followed by the body of the function containing the executable code for that function.

Function Definition

Function blocks starts with the keyword **def**.

- The keyword is followed by the function name and parentheses **(())**.
- After the parentheses a colon **(:)** is placed.
- Parameters or arguments that the function accept are placed within parentheses.
- The first statement of a function can be an optional statement - the **docstring** describe what the function does.
- The code block within the function is properly indented to form the block code.
- A function may have a **return[expression]** statement. That is, the return statement is optional.
- You can assign the function name to a variable. Doing this will allow you to call same function using the name of that variable.

Example:

```
def diff(x,y):          # function to subtract two numbers
    return x-y
a = 20
b = 10
operation = diff         # function name assigned to a variable
print(operation(a,b))    # function called using variable name

OUTPUT
10
```

Function Definition

```
def diff(x,y):          # function to subtract two numbers
    return x-y

a = 20
b = 10
operation = diff        # function name assigned to a variable
print(operation(a,b))  # function called using variable name
```

OUTPUT

10

Function Call

The function call statement invokes the function. When a function is invoked the program control jumps to the called function to execute the statements that are a part of that function. Once the called function is executed, the program control passes back to the calling function.

Function Parameters

A function can take parameters which are nothing but some values that are passed to it so that the function can manipulate them to produce the desired result. These parameters are normal variables with a small difference that the values of these variables are defined (initialized) when we call the function and are then passed to the function.

Function name and the number and type of arguments in the function call must be same as that given in the function definition.

If the data type of the argument passed does not matches with that expected in function then an error is generated. Example:

```
def function_name(variable1, variable2,..)
    documentation string
    statement block
    return [expression]
```

Function Header

Function Body

Examples

```
def func():  
    for i in range(4):  
        print("Hello World")  
func()      #function call
```

OUTPUT

```
Hello World  
Hello World  
Hello World  
Hello World
```

```
def func(i):  
    print("Hello World", i)  
func(5+2*3)
```

OUTPUT

```
Hello World 11
```

```
def total(a,b):      # function accepting parameters  
    result = a+b  
    print("Sum of ", a, " and ", b, " = ", result)
```

```
a = int(input("Enter the first number : "))  
b = int(input("Enter the second number : "))  
total(a,b) #function call with two arguments
```

OUTPUT

```
Enter the first number : 10  
Enter the second number : 20  
Sum of 10 and 20 = 30
```

```
def func(i):          # function definition header accepts a variable with name i  
    print("Hello World", i)  
    j = 10  
    func(j)           # Function is called using variable j
```

OUTPUT

```
Hello World 10
```

Local and Global Variables

A variable which is defined within a function is **local** to that function. A local variable can be accessed from the point of its definition until the end of the function in which it is defined. It exists as long as the function is executing. Function parameters behave like local variables in the function. Moreover, whenever we use the assignment operator (=) inside a function, a new local variable is created.

Global variables are those variables which are defined in the main body of the program file. They are visible throughout the program file. As a good programming habit, you must try to avoid the use of global variables because they may get altered by mistake and then result in erroneous output.

Local and Global Variables

Example:

```
num1 = 10    # global variable
print("Global variable num1 = ", num1)
def func(num2):    # num2 is function parameter
    print("In Function - Local Variable num2 = ", num2)
    num3 = 30      # num3 is a local variable
    print("In Function - Local Variable num3 = ", num3)
func(20)          # 20 is passed as an argument to the function
print("num1 again = ", num1)    # global variable is being accessed
#Error- local variable can't be used outside the function in which it is defined
print("num3 outside function = ", num3)
```

OUTPUT

```
Global variable num1 = 10
In Function - Local Variable num2 = 20
In Function - Local Variable num3 = 30
num1 again = 10
num3 outside function =
Traceback (most recent call last):
  File "C:\Python34\Try.py", line 12, in <module>
    print("num3 outside function = ", num3)
NameError: name 'num3' is not defined
```

Programming Tip: Variables can only be used after the point of their declaration

Using the Global Statement

To define a variable defined inside a function as global, you must use the global statement. This declares the local or the inner variable of the function to have module scope.

Example:

```
var = "Good"
def show():
    global var1
    var1 = "Morning"
    print("In Function var is - ", var)

show()
print("Outside function, var1 is - ", var1)    #accessible as it is global
variable
print("var is - ", var)
```

OUTPUT

```
In Function var is -  Good
Outside function, var1 is -  Morning
var is -  Good
```

Programming Tip: All variables have the scope of the block.

Resolution of names

Scope defines the visibility of a name within a block. If a local variable is defined in a block, its scope is that particular block. If it is defined in a function, then its scope is all blocks within that function.

When a variable name is used in a code block, it is resolved using the nearest enclosing scope. If no variable of that name is found, then a **NameError** is raised. In the code given below, `str` is a global string because it has been defined before calling the function.

Example:

```
def func():  
    print(str)  
str = "Hello World !!!"  
func()
```

OUTPUT

```
Hello World !!!
```

The Return Statement

The syntax of return statement is,

return [expression]

The expression is written in brackets because it is optional. If the expression is present, it is evaluated and the resultant value is returned to the calling function. However, if no expression is specified then the function will return **None**.

The return statement is used for two things.

- Return a value to the caller
- To end and exit a function and go back to its caller

Example:

```
def cube(x):  
    return (x*x*x)  
num = 10  
result = cube(num)  
print('Cube of ', num, ' = ', result)
```

OUTPUT

Cube of 10 = 1000

Required Arguments

In the *required arguments* , the arguments are passed to a function in correct positional order. Also, the number of arguments in the function call should exactly match with the number of arguments specified in the function definition

Examples:

```
def display():  
    print "Hello"  
display("Hi")
```

OUTPUT

```
TypeError: display() takes  
no arguments (1 given)
```

```
def display(str):  
    print str  
display()
```

OUTPUT

```
TypeError: display() takes  
exactly 1 argument (0  
given)
```

```
def display(str):  
    print str  
str ="Hello"  
display(str)
```

OUTPUT

```
Hello
```

Keyword Arguments

When we call a function with some values, the values are assigned to the arguments based on their position. Python also allow functions to be called using keyword arguments in which the order (or position) of the arguments can be changed. The values are not assigned to arguments according to their position but based on their name (or keyword).

Keyword arguments are beneficial in two cases.

an alternative to positional arguments is keyword argument, if a programmer knows the parameter name used within the function then they can explicitly use the parameter name while calling the function. a programmer can pass a keyword argument to a function by using its corresponding parameter name rather than its position.

- First, if you skip arguments.
- Second, if in the function call you change the order of parameters.

Example:

```
def display(str, int_x, float_y):  
    print("The string is : ",str)  
    print("The integer value is : ", int_x)  
    print("The floating point value is : ", float_y)  
display(float_y = 56789.045, str = "Hello", int_x = 1234)
```

OUTPUT

```
The string is: Hello  
The integer value is: 1234  
The floating point value is: 56789.045
```

Variable-length Arguments

In some situations, it is not known in advance how many arguments will be passed to a function. In such cases, Python allows programmers to make function calls with arbitrary (or any) number of arguments.

When we use arbitrary arguments or variable length arguments, then the function definition use an asterisk (*) before the parameter name. The syntax for a function using variable arguments can be given as,

```
def functionname([arg1, arg2,... ] *var_args_tuple ):
    function statements
    return [expression]
```

Example:

```
def func(name, *fav_subjects):
    print("\n", name, " likes to read ")
    for subject in fav_subjects:
        print(subject)
func("Goransh", "Mathematics", "Android Programming")
func("Richa", "C", "Data Structures", "Design and Analysis of Algorithms")
func("Krish")
```

OUTPUT

```
Goransh likes to read  Mathematics Android Programming
Richa likes to read  C Data Structures Design and Analysis of Algorithms
Krish likes to read
```

Default Arguments

Python allows users to specify function arguments that can have default values. This means that a function can be called with fewer arguments than it is defined to have. That is, if the function accepts three parameters, but function call provides only two arguments, then the third parameter will be assigned the default (already specified) value.

The default value to an argument is provided by using the assignment operator (=). Users can specify a default value for one or more arguments.

Example:

```
def display(name, course = "BTech"):
    print("Name : " + name)
    print("Course : ", course)
display(course = "BCA", name = "Arav") # Keyword Arguments
display(name = "Reyansh")              # Default Argument for course
```

OUTPUT

```
Name : Arav
Course :  BCA
Name : Reyansh
Course :  BTech
```

Lambda Functions Or Anonymous Functions

Lambda or anonymous functions are so called because they are not declared as other functions using the `def` keyword. Rather, they are created using the `lambda` keyword. Lambda functions are throw-away functions, i.e. they are just needed where they have been created and can be used anywhere a function is required.

The `lambda` feature was added to Python due to the demand from LISP programmers.

Lambda functions contain only a single line. Its syntax can be given as,

```
lambda arguments: expression
```

Example:

```
sum = lambda x, y: x + y  
print("Sum = ", sum(3, 5))
```

OUTPUT

```
Sum = 8
```

Documentation Strings

Docstrings (documentation strings) serve the same purpose as that of comments, as they are designed to explain code. However, they are more specific and have a proper syntax.

```
def functionname(parameters):  
    "function_docstring"  
    function statements  
    return [expression]
```

Example:

```
def func():  
    """The program just prints a message.  
    It will display Hello World !!! """  
    print("Hello World !!!")  
print(func.__doc__)
```

OUTPUT

```
The program just prints a message.  
It will display Hello World !!!
```

Recursive Functions

A recursive function is defined as a function that calls itself to solve a smaller version of its task until a final call is made which does not require a call to itself. Every recursive solution has two major cases, which are as follows:

- **base case**, in which the problem is simple enough to be solved directly without making any further calls to the same function.
- **recursive case**, in which first the problem at hand is divided into simpler sub parts.

Recursion utilized divide and conquer technique of problem solving.

Example:

```
def fact(n):  
    if(n==1 or n==0):  
        return 1  
    else:  
        return n*fact(n-1)  
n = int(input("Enter the value of n : "))  
print("The factorial of",n,"is",fact(n))
```

OUTPUT

```
Enter the value of n : 6  
The factorial of 6 is 720
```

The from...import Statement

A module may contain definition for many variables and functions. When you import a module, you can use any variable or function defined in that module. But if you want to use only selected variables or functions, then you can use the from...import statement. For example, in the aforementioned program you are using only the path variable in the sys module, so you could have better written from sys import path.

Example:

```
from math import pi
print("PI = ", + pi)
```

OUTPUT

```
3.141592653589793
```

To import more than one item from a module, use a comma separated list. For example, to import the value of pi and sqrt() from the math module you can write,

```
from math import pi, sqrt
```

Making your own Modules

Every Python program is a module, that is, every file that you save as .py extension is a module.

Examples:

First write these lines in a file and save the file as MyModule.py

```
def display():          #function definition
    print("Hello")
    print("Name of called module is : ", __name__)

str = "Welcome to the world of Python !!!"    #variable definition
```

Then, open another file (main.py) and write the lines of code given below.

```
import MyModule
print("MyModule str = ", MyModule.str)        #using variable defined in MyModule
MyModule.display()                            #using function defined in MyModule
print("Name of calling module is : ", __name__)
```

When you run this code, you will get the following output.

```
MyModule str =  Welcome to the world of Python !!!
Hello
Name of called module is :  MyModule
Name of calling module is :  __main__
```

The dir() function

dir() is a built-in function that lists the identifiers defined in a module. These identifiers may include functions, classes and variables. If no name is specified, the dir() will return the list of names defined in the current module.

Example:

```
def print_var(x):  
    print(x)  
x = 10  
print_var(x)  
print(dir())
```

OUTPUT

```
10  
['__builtins__', '__doc__', '__file__', '__name__', '__package__', 'print_var', 'x']
```

Modules and Namespaces

A **namespace** is a container that provides a named context for identifiers. Two identifiers with the same name in the same scope will lead to a name clash. In simple terms, Python does not allow programmers to have two different identifiers with the same name. However, in some situations we need to have same name identifiers. To cater to such situations, namespaces is the keyword. Namespaces enable programs to avoid potential name clashes by associating each identifier with the namespace from which it originates.

Example:

```
# module1
def repeat_x(x):
    return x*2

# module2
def repeat_x(x):
    return x**2

import module1
import module2
result = repeat_x(10)    # ambiguous reference for identifier repeat_x
```



Local, Global, and Built-in Namespaces

During a program's execution, there are three main namespaces that are referenced- the built-in namespace, the global namespace, and the local namespace. The built-in namespace, as the name suggests contains names of all the built-in functions, constants, etc that are already defined in Python. The global namespace contains identifiers of the currently executing module and the local namespace has identifiers defined in the currently executing function (if any).

When the Python interpreter sees an identifier, it first searches the local namespace, then the global namespace, and finally the built-in namespace. Therefore, if two identifiers with same name are defined in more than one of these namespaces, it becomes masked.

Local, Global, and Built-in Namespaces

Example:

```
def max(numbers):      # global namespace
    print("USER DEFINED FUNCTION MAX.....")
    large = -1         # local namespace
    for i in numbers:
        if i>large:
            large = i
    return large
numbers = [9,-1,4,2,7]
print(max(numbers))
print("Sum of these numbers = ", sum(numbers)) #built in namespace
```

OUTPUT

```
USER DEFINED FUNCTION MAX.....
```

```
9
```

```
Sum of these numbers = 21
```

Module Private Variables

In Python, all identifiers defined in a module are public by default. This means that all identifiers are accessible by any other module that imports it. But, if you want some variables or functions in a module to be privately used within the module, but not to be accessed from outside it, then you need to declare those identifiers as private.

In Python identifiers whose name starts with two underscores (__) are known as private identifiers. These identifiers can be used only within the module. In no way, they can be accessed from outside the module. Therefore, when the module is imported using the **import * from modulename**, *all the identifiers of a module's namespace is imported except the private ones (ones beginning with double underscores). Thus, private identifiers become inaccessible from within the importing module.*

Packages in Python

A package is a hierarchical file directory structure that has modules and other packages within it. Like modules, you can very easily create packages in Python.

Every package in Python is a directory which must have a special file called `__init__.py`. This file may not even have a single line of code. It is simply added to indicate that this directory is not an ordinary directory and contains a Python package. In your programs, you can import a package in the same way as you import any module.

For example, to create a package called **MyPackage**, create a directory called **MyPackage** having the module **MyModule** and the `__init__.py` file. Now, to use **MyModule** in a program, you must first import it. This can be done in two ways.

```
import MyPackage.MyModule
```

or

```
from MyPackage import MyModule
```

Globals(), Locals(), And Reload()

The **globals()** and **locals()** functions are used to return the names in the global and local namespaces (In Python, each function, module, class, package, etc owns a “namespace” in which variable names are identified and resolved). The result of these functions is of course, dependent on the location from where they are called. For example,

If **locals()** is called from within a function, names that can be accessed locally from that function will be returned.

If **globals()** is called from within a function, all the names that can be accessed globally from that function is returned.

Reload()- When a module is imported into a program, the code in the module is executed only once. If you want to re-execute the top-level code in a module, you must use the **reload()** function. This function again imports a module that was previously imported.



Python Strings

Strings

Python treats strings as contiguous series of characters delimited by single, double or even triple quotes. Python has a built-in string class named "str" that has many useful features. We can simultaneously declare and define a string by creating a variable of string type. This can be done in several ways which are as follows:

```
name = "India" graduate = 'N' country = name nationality = str("Indian")
```

Indexing: Individual characters in a string are accessed using the subscript ([]) operator. The expression in brackets is called an index. The index specifies a member of an ordered set and in this case it specifies the character we want to access from the given set of characters in the string.

The index of the first character is 0 and that of the last character is n-1 where n is the number of characters in the string. If you try to exceed the bounds (below 0 or above n-1), then an error is raised.

Strings

Traversing a String: A string can be traversed by accessing character(s) from one index to another. For example, the following program uses indexing to traverse a string from first character to the last.

Example:

```
message = "Hello!"
index = 0
for i in message:
    print("message[", index, "] = ", i)
    index += 1
```

OUTPUT

```
message[ 0 ] = H
message[ 1 ] = e
message[ 2 ] = l
message[ 3 ] = l
message[ 4 ] = o
message[ 5 ] = !
```

Concatenating, Appending and Multiplying Strings

Examples:

```
str1 = "Hello "  
str2 = "World"  
str3 = str1 + str2  
print("The concatenated string is : ", str3)
```

OUTPUT

```
The concatenated string is : Hello World
```

```
str = "Hello"  
print(str * 3)
```

OUTPUT

```
Hello Hello Hello
```

```
str = "Hello, "  
name = input("\n Enter your name : ")  
str += name  
str += ". Welcome to Python Programming."  
print(str)
```

OUTPUT

```
Enter your name : Arnav  
Hello, Arnav. Welcome to Python Programming.
```

Strings are Immutable

Python strings are immutable which means that once created they cannot be changed. Whenever you try to modify an existing string variable, a new string is created.

Example:

```
str1 = "Hello"
print("Str1 is : ", str1)
print("ID of str1 is : ", id(str1))

str2 = "World"
print("Str2 is : ", str2)
```

```
print("ID of str1 is : ", id(str2))
str1 += str2
print("Str1 after concatenation is : ", str1)
print("ID of str1 is : ", id(str1))
str3 = str1
print("str3 = ", str3)
print("ID of str3 is : ", id(str3))
```

OUTPUT

```
Str1 is : Hello
ID of str1 is : 45093344
Str2 is : World
ID of str1 is : 45093312
Str1 after concatenation is : HelloWorld
ID of str1 is : 43861792
str3 = HelloWorld
ID of str3 is : 43861792
```

String Formatting Operator

The % operator takes a format string on the left (that has %d, %s, etc) and the corresponding values in a tuple (will be discussed in subsequent chapter) on the right. The format operator, % allow users to construct strings, replacing parts of the strings with the data stored in variables. The syntax for the string formatting operation is:

"<Format>" % (<Values>)

Example:

```
name = "Aarish"  
age = 8  
print("Name = %s and Age = %d" %(name, age))  
print("Name = %s and Age = %d" %("Anika", 6))
```

OUTPUT

```
Name = Aarish and Age = 8  
Name = Anika and Age = 6
```

Built-in String Methods and Functions

Function	Usage	Example
<code>capitalize()</code>	This function is used to capitalize first letter of the string.	<pre>str = "hello" print(str.capitalize())</pre> OUTPUT Hello
<code>center(width, fillchar)</code>	Returns a string with the original string centered to a total of width columns and filled with fillchar in columns that do not have characters.	<pre>str = "hello" print(str.center(10, '*'))</pre> OUTPUT **hello**
<code>count(str, beg, end)</code>	Counts number of times str occurs in a string. You can specify beg as 0 and end as the length of the message to search the entire string or use any other value to just search a part of the string.	<pre>str = "he" message = "helloworldhellohello" print(message.count(str,0, len(message)))</pre> OUTPUT 3
<code>endswith(suffix, beg, end)</code>	Checks if string ends with suffix; returns True if so and False otherwise. You can either set beg = 0 and end equal to the length of the message to search entire string or use any other value to search a part of it.	<pre>message = "She is my best friend" print(message.endswith("end", 0, len(message)))</pre> OUTPUT True

Built-in String Methods and Functions

<code>find(str, beg, end)</code>	Checks if <code>str</code> is present in <code>string</code> . If found it returns the position at which <code>str</code> occurs in <code>string</code> , otherwise returns <code>-1</code> . You can either set <code>beg = 0</code> and <code>end</code> equal to the length of the message to search entire string or use any other value to search a part of it.	<pre>message = "She is my best friend" print(message.find("my", 0, len(message)))</pre> OUTPUT 7
<code>index(str, beg, end)</code>	Same as <code>find</code> but raises an exception if <code>str</code> is not found.	<pre>message = "She is my best friend" print(message.index("mine", 0, len(message)))</pre> OUTPUT ValueError: substring not found
<code>rfind(str, beg, end)</code>	Same as <code>find</code> but starts searching from the end.	<pre>str = "Is this your bag?" print(str.rfind("is", 0, len(str)))</pre> OUTPUT 5
<code>rindex(str, beg, end)</code>	Same as <code>rindex</code> but start searching from the end and raises an exception if <code>str</code> is not found.	<pre>str = "Is this your bag?" print(str.rindex("you", 0, len(str)))</pre> OUTPUT 8

Built-in String Methods and Functions

<code>isalnum()</code>	Returns True if string has at least 1 character and every character is either a number or an alphabet and False otherwise.	<pre>message = "JamesBond007" print(message.isalnum())</pre> OUTPUT True
<code>isalpha()</code>	Returns True if string has at least 1 character and every character is an alphabet and False otherwise.	<pre>message = "JamesBond007" print(message.isalpha())</pre> OUTPUT False
<code>isdigit()</code>	Returns True if string contains only digits and False otherwise.	<pre>message = "007" print(message.isdigit())</pre> OUTPUT True
<code>islower()</code>	Returns True if string has at least 1 character and every character is a lowercase alphabet and False otherwise.	<pre>message = "Hello" print(message.islower())</pre> OUTPUT False
<code>isspace()</code>	Returns True if string contains only whitespace characters and False otherwise.	<pre>message = " " print(message.isspace())</pre> OUTPUT True
<code>isupper()</code>	Returns True if string has at least 1 character and every character is an upper case alphabet and False otherwise.	<pre>message = "HELLO" print(message.isupper())</pre> OUTPUT True
<code>len(string)</code>	Returns the length of the string.	<pre>str = "Hello" print(len(str))</pre> OUTPUT 5
<code>ljust(width[, fillchar])</code>	Returns a string left-justified to a total of width columns. Columns without characters are padded with the character specified in the <code>fillchar</code> argument.	<pre>str = "Hello" print(str.ljust(10, '*'))</pre> OUTPUT Hello*****
<code>rjust(width[, fillchar])</code>	Returns a string right-justified to a total of width columns. Columns without characters are padded with the character specified in the <code>fillchar</code> argument.	<pre>str = "Hello" print(str.rjust(10, '*'))</pre> OUTPUT *****Hello

Slice Operation

A substring of a string is called a *slice*. The slice operation is used to refer to sub-parts of sequences and strings. You can take subset of string from original string by using `[ ]` operator also known as *slicing operator*.

Index from
the start

P	Y	T	H	O	N
0	1	2	3	4	5
-6	-5	-4	-3	-2	-1

Index from
the end

Examples:

```
str = "PYTHON"
print("str[1:5] = ", str[1:5])    #characters starting at index 1 and extending up
to but not including index 5
print("str[:6] = ", str[:6])      # defaults to the start of the string
print("str[1:] = ", str[1:])      # defaults to the end of the string
print("str[:] = ", str[:])        # defaults to the entire string
print("str[1:20] = ", str[1:20])  # an index that is too big is truncated down to
length of the string
```

OUTPUT

```
str[1:5] =  YTHO
str[:6] =  PYTHON
str[1:] =  YTHON
str[:] =  PYTHON
str[1:20] =  YTHON
```

Programming Tip: Python does not have any separate data type for characters. They are represented as a single character string.

Specifying Stride while Slicing Strings

In the slice operation, you can specify a third argument as the **stride**, which refers to the number of characters to move forward after the first character is retrieved from the string. By default the value of stride is 1, so in all the above examples where he had not specified the stride, it used the value of 1 which means that every character between two index numbers is retrieved.

Examples:

```
str = "Welcome to the world of Python"
print("str[2:10] = ", str[2:10])    # default stride is 1
print("str[2:10:1] = ", str[2:10:1]) # same as stride = 1
print("str[2:10:2] = ", str[2:10:2]) # skips every alternate character
print("str[2:13:4] = ", str[2:13:4]) # skips every fourth character
```

OUTPUT

```
str[2:10] = lcome to
str[2:10:1] = lcome to
str[2:10:2] = loet
str[2:13:4] = le
```

ord() and chr() Functions

ord() function returns the **ASCII** code of the character and **chr()** function returns character represented by a **ASCII** number. Examples:

```
ch = 'R'
print(ord(ch))
```

OUTPUT
82

```
print(chr(82))
```

OUTPUT
R

```
print(chr(112))
```

OUTPUT
p

```
print(ord('p'))
```

OUTPUT
112

in and not in Operators

in and not in operators can be used with strings to determine whether a string is present in another string. Therefore, the in and not in operator are also known as membership operators.

Examples:

```
str1 = "Welcome to the world of Python  
!!!"
str2 = "the"
if str2 in str1:
    print("Found")
else:
    print("Not Found")
```

OUTPUT
Found

```
str1 = "This is a very good book"
str2 = "best"
if str2 not in str1:
    print("The book is very good but it  
may not be the best one.")
else:
    print("It is the best book.")
```

OUTPUT
The book is very good but it may not be
the best one.

Comparing Strings

Operator	Description	Example
==	If two strings are equal, it returns True.	>>> "AbC" == "AbC" True
!= or <>	If two strings are not equal, it returns True.	>>> "AbC" != "Abc" True >>> "abc" <> "ABC" True
>	If the first string is greater than the second, it returns True.	>>> "abc" > "Abc" True
<	If the second string is greater than the first, it returns True.	>>> "abC" < "abc" True
>=	If the first string is greater than or equal to the second, it returns True.	>>> "aBC" >= "ABC" True
<=	If the second string is greater than or equal to the first, it returns True.	>>> "ABc" <= "ABc" True

Iterating String

String is a sequence type (sequence of characters). You can iterate through the string using for loop.

Examples:

```
str = "Welcome to Python"
for i in str:
    print(i, end=' ')
```

OUTPUT

W e l c o m e t o P y t h o n

```
message = " Welcome to Python "
index = 0
while index < len(message):
    letter = message[index]
    print(letter, end=' ')
    index += 1
```

OUTPUT

W e l c o m e t o P y t h o n

The String Module

The string module consist of a number of useful constants, classes and functions (some of which are deprecated). These functions are used to manipulate strings.

Examples:

```
str = "Welcome to the world of Python"
print("Uppercase - ", str.upper())
print("Lowercase - ", str.lower())
print("Split - ", str.split())
print("Join - ", '-'.join(str.split()))
print("Replace - ", str.replace("Python", "Java"))
print("Count of o - ", str.count('o') )
print("Find of - ", str.find("of"))
```

OUTPUT

```
Uppercase -  WELCOME TO THE WORLD OF PYTHON
Lowercase -  welcome to the world of python
Split -  ['Welcome', 'to', 'the', 'world', 'of', 'Python']
Join -  Welcome-to-the-world-of-Python
Replace -  Welcome to the world of Java
Count of o -  5
Find of -  21
```

Programming Tip: A method is called by appending its name to the variable name using the period as a delimiter.

Working with Constants in String Module

You can use the constants defined in the string module along with the find function to classify characters. For example, if `find(lowercase, ch)` returns a value except -1, then it means that `ch` must be a lowercase character. An alternate way to do the same job is to use the `in` operator or even the comparison operation.

Examples:

```
# First Way
import string
print(string.find(string.
lowercase, 'g') != -1)
```

OUTPUT

True

```
# Second Way
import string
print('g' in string.
lowercase)
```

OUTPUT

True

```
# Third Way
import string
ch = 'g'
print('a' <= ch <= 'z')
```

OUTPUT

True

Regular Expressions

Regular Expressions are a powerful tool for various kinds of string manipulation. These are basically a special text string that is used for describing a search pattern to extract information from text such as code, files, log, spreadsheets, or even documents.

Regular expressions are a domain specific language (DSL) that is present as a library in most of the modern programming languages, besides Python. A *regular expression* is a special sequence of characters that helps to match or find strings in another string. In Python, regular expressions can be accessed using the `re` module which comes as a part of the Standard Library

The Match Function

As the name suggest, the `match()` function matches a pattern to string with optional flags. The syntax of `match()` function is,

`re.match (pattern, string, flags=0)`

EXAMPLE- Match() Function

Examples:

Flag	Description
re.I	Case sensitive matching
re.M	Matches at the end of the line
re.X	Ignores whitespace characters
re.U	Interprets letters according to Unicode character set

```
import re
string = "She sells sea shells on the sea shore"
pattern1 = "sells"
if re.match(pattern1, string):
    print("Match Found")
else:
    print(pattern1, "is not present in the string")
pattern2 = "She"
if re.match(pattern2, string):
    print("Match Found")
else:
    print(pattern2, "is not present in the string")
```

OUTPUT

```
sells is not present in the string
Match Found
```

The search Function

The **search()** function in the re module searches for a pattern anywhere in the string. Its syntax can be given as, **re.search(pattern, string, flags=0)**

The syntax is similar to the match() function. The function searches for first occurrence of *pattern* within a *string* with optional *flags*. If the search is successful, a *match* object is returned and None otherwise.

Example:

```
import re
string = "She sells sea shells on the sea shore"
pattern = "sells"
if re.search(pattern, string):
    print("Match Found")
else:
    print(pattern, "is not present in the string")
```

OUTPUT

Match Found

The sub() Function

The **sub()** function in the re module can be used to search a pattern in the string and replace it with another pattern. The syntax of sub() function can be given as, **re.sub(pattern, repl, string, max=0)**

According to the syntax, the sub() function replaces all occurrences of the pattern in string with repl, substituting all occurrences unless any max value is provided. This method returns modified string.

Example:

```
import re
string = "She sells sea shells on the sea shore"
pattern = "sea"
repl = "ocean"
new_string = re.sub(pattern, repl, string, 1)
print(new_string)
```

OUTPUT

```
She sells ocean shells on the sea shore
```

The findall() and finditer() Function

The **findall()** function is used to search a string and returns a list of matches of the pattern in the string. If no match is found, then the returned list is empty. The syntax of match() function can be given as,

matchList = re.findall(pattern, input_str, flags=0)

Examples:

```
import re
pattern = r"[a-zA-Z]+ \d+"
matches = re.findall(pattern, "LXI 2013, VXi 2015, VDI 20104, Maruti Suzuki Cars in Inida")
for match in matches:
    print(match, end = " ")
```

OUTPUT

```
LXI 2013 VXi 2015 VDI 20104
```

Flag Options

The `search()`, `findall()` and `match()` functions of the module take options to modify the behavior of the pattern match. Some of these flags are:

re.I or re.IGNORECASE — Ignores case of characters, so "Match", "MATCH", "mAtCh", etc are all same

re.S or re.DOTALL — Enables dot (.) to match newline. By default, dot matches any character other than the newline character.

re.M or re.MULTILINE — Makes the ^ and \$ to match the start and end of each line. That is, it matches even after and before line breaks in the string. By default, ^ and \$ matches the start and end of the whole string.

re.L or re.LOCALE- Makes the flag \w to match all characters that are considered letters in the given current locale settings.

re.U or re.UNICODE- Treats all letters from all scripts as word characters.

Metacharacters in Regular Expression

Metacharacter	Description	Example	Remarks
^	Matches at the beginning of the line.	^Hi	It will match Hi at the start of the string.
\$	Matches at the end of the line.	Hi\$	It will match Hi at the end of the string.
.	Matches any single character except the newline character.	Lo.	It will match Lot, Log, etc.
[...]	Matches any single character in brackets.	[Hh]ello	It will match "Hello" or "hello".
[^...]	Matches any single character not in brackets.	[^aeiou]	It will match anything other than a lowercase vowel.
re*	Matches 0 or more occurrences of regular expression.	[a-z]*	It will match zero or more occurrence of lowercase characters.
re+	Matches 1 or more occurrence of regular expression.	[a-z]+	It will match one or more occurrence of lowercase characters
re?	Matches 0 or 1 occurrence of regular expression.	Book?	It will match "Book" or "Books".
re{n}	Matches exactly n number of occurrences of regular expression.	42{1}5	It will match 425.
re{n,}	Matches n or more occurrences of regular expression.	42{1,}5	It will match 42225 or any number with more than one 2s between 4 and 5.
re{n,m}	Matches at least n and at most m occurrences of regular expression.	42{1,3}5	It will match 425, 4225, 42225.

Metacharacters in Regular Expression

<code>a b</code>	Matches either a or b.	<code>"Hello" "Hi"</code>	It will match Hello or Hi.
<code>\w</code>	Matches word characters.	<code>re.search(r'\w', 'xx123xx')</code>	Match will be made.
<code>\W</code>	Matches non-word characters.	<code>if(re.search(r'\W', '@#\$%')): print("Done")</code>	Done
<code>\s</code>	Matches whitespace, equivalent to <code>[\t\n\r\f]</code> .	<code>if(re.search(r'\s', "abcdsd")): print("Done")</code>	Done
<code>\S</code>	Matches non-whitespace, equivalent to <code>[\t\n\r\f]</code> .	<code>if(re.search(r'\S', "abcdsd")): print("Done")</code>	Done
<code>\d{n}</code>	Matches exactly n digits.	<code>\d{2}</code>	It will match exactly 2 digits.
<code>\d{n,}</code>	Matches n or more digits.	<code>\d{3,}</code>	It will match 3 or more digits.
<code>\d{n,m}</code>	Matches n and at most m digits.	<code>\d{2,4}</code>	It will match 2,3 or 4 digits.
<code>\D</code>	Matches non-digits.	<code>(\D+\d)</code>	It will match Hello 5678, or any string starting with no digit followed by digits(s).
<code>\A</code>	Matches beginning of the string.	<code>\AHi</code>	It will match Hi at the beginning of the string.
<code>\Z</code>	Matches end of the string.	<code>Hi\Z</code>	It will match Hi at the end of the string.
<code>\G</code>	Matches point where last match finished.	<code>import re if(re.search(r'\Gabc', 'abcbaabc')): print("Done") else: print("Not Done")</code>	Not Done
<code>\b</code>	Matches word boundaries when outside brackets. Matches backspace when inside brackets.	<code>\bHi\b</code>	It will match Hi at the word boundary.
<code>\B</code>	Matches non-word boundaries.	<code>\bHi\B</code>	Hi should start at word boundary but end at a non-boundary as in High
<code>\n, \t, etc.</code>	Matches newlines, tabs, etc.	<code>re.search(r'\t', '123 \t abc')</code>	Match will be made.

Character Classes

When we put the characters to be matched inside square brackets, we call it a **character class**. For example, `[aeiou]` defines a character class that has a vowel character.

Examples:

```
import re
pattern=r"[aeiou]"
if re.search(pattern,"clue"):
    print("Match clue")
if re.search(pattern,"bcdfg"):
    print("Match bcdfg")
```

OUTPUT

Match clue

Groups

A **group** is created by surrounding a part of the regular expression with parentheses. You can even give group as an argument to the metacharacters such as * and ?.

Example:

```
import re
pattern = r"gr(ea)*t"  # group of ea created
if re.match(pattern, "great"):
    print("Match ea")
if re.match(pattern, "greaeaeaeaeaeaeat"):
    print("Match greaeaeaeaeaeaeat")
```

OUTPUT

```
Match ea
Match greaeaeaeaeaeaeat
```

The content of groups in a match can be accessed by using the `group()` function. For example,

- `group(0)` or `group()` returns the whole match.
- `group(n)`, where `n` is greater than 0, returns the `n`th group from the left.
- `group()` returns all groups up from 1.